

Technology Stack & Architecture

A complete technical overview of the frameworks, languages, and infrastructure powering the Mycel ecosystem — custom blockchain, backend API, social platform, chat, and mobile.

Rust

Substrate

Axum

Next.js

PostgreSQL

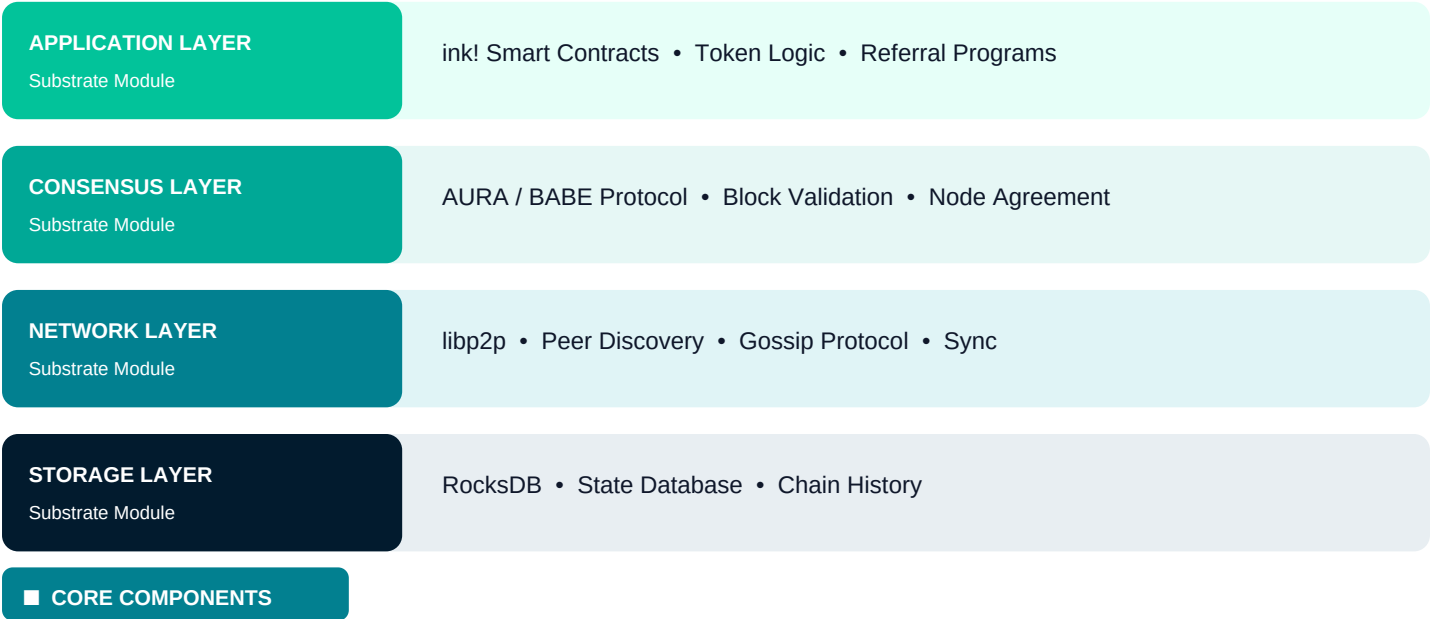
React Native

Redis

Docker

Mycelium is a purpose-built blockchain giving the ecosystem full control over token logic, transaction rules, and economic incentives. Built in Rust using the Substrate framework.

MYCELIUM CHAIN ARCHITECTURE



Component	Technology	Purpose
Blockchain Framework	Substrate (Rust)	Build and run the Mycelium chain
P2P Networking	libp2p	Nodes discover and talk to each other
Consensus Algorithm	AURA / BABE (Substrate)	How new blocks get agreed upon & added
Smart Contracts	ink!	Rust-based contracts: tokens, referrals
State Storage	RocksDB	Stores all blockchain state on disk
Async Runtime	Tokio	Handles concurrent node operations

NOTE

Substrate is the same framework used to build Polkadot — battle-tested, fully Rust-native, and designed for custom chains.

TIMELINE IMPACT

Building a custom blockchain adds 6–12 months to the roadmap. Substrate accelerates this significantly, but confirm this trade-off vs. deploying on Solana first and migrating later in the kickoff meeting.

■ BACKEND API (RUST)

Component	Technology	Purpose
Web Framework	Axum (Rust)	Primary REST + WebSocket API server
HTTP Client	reqwest	Outbound HTTP calls to external services
Serialization	serde / serde_json	Fast JSON serialization for API payloads
Auth — Web2	JWT (jsonwebtoken)	Standard session auth for regular users
Auth — Web3	Wallet signature verify	Cryptographic proof of wallet ownership
Validation	validator	Input sanitization and schema validation
Error Handling	anyhow / thiserror	Structured, traceable error management

■ DATABASE LAYER

Component	Technology	Purpose
Primary Database	PostgreSQL	Users, posts, referrals, platform data
ORM / Query Builder	sqlx (async Rust)	Type-safe, async SQL queries in Rust
Cache / Sessions	Redis	Token sessions, rate limiting, fast reads
Search	Meilisearch	Full-text search across posts & communities
Migrations	sqlx-cli	Versioned schema migrations in Rust

■ ASYNC PROCESSING

Component	Technology	Purpose
Message Queue	RabbitMQ	Async token transaction processing
Background Jobs	Tokio tasks	Referral reward calculations, notifications
Rate Limiting	Redis + tower-governor	API rate limiting per user / wallet

■ MULTI-ROLE CHAT APP

Component	Technology	Purpose
Real-Time Protocol	WebSockets (tokio-tungstenite)	Live messaging between users
Room Management	Redis pub/sub	Managing active chat rooms at scale
Media (future)	Cloudflare R2	File/image storage for chat (Phase 3+)

FRONTEND (WEB)

DEVOPS & INFRA

Layer	Technology
Framework	Next.js 14 (React)
Language	TypeScript
Styling	TailwindCSS
State Mgmt	Zustand
Chain Bridge	Polkadot.js API
Auth UI	Wallet connect modal
Forms	React Hook Form
MOBILE APP	

Layer	Technology
Framework	React Native
Navigation	React Navigation
State	Zustand (shared)
Wallet	WalletConnect
Push Notifs	Expo Notifications

Layer	Technology
Containers	Docker
Orchestration	Kubernetes
CI/CD	GitHub Actions
Blockchain Nodes	Self-hosted VPS
Hosting	AWS / DigitalOcean
CDN	Cloudflare
Monitoring	Grafana + Prometheus
Logging	Loki

Recommended Build Order

01

Mycelium Blockchain

02

Rust Backend API

03

Next.js Frontend

04

WebSocket Chat

05

React Native App

06

DevOps & Scale